

SQL CASE STUDY

Supply Chain Data Analysis

1. Overview & Learning Objectives

In this case study, you will work as a junior data analyst for GlobalTrade Co., a mid-sized manufacturing and distribution company. Your job is to query the company's supply chain database to answer critical business questions — covering inventory management, supplier performance, order fulfillment, and logistics costs.

By completing this assignment, you will be able to:

- Write complex multi-table SQL queries using various JOIN types
- Apply aggregate functions, GROUP BY, HAVING, and window functions
- Use subqueries and Common Table Expressions (CTEs)
- Analyze supply chain KPIs such as lead time, fill rate, and inventory turnover
- Identify supplier bottlenecks and inventory anomalies using SQL
- Build ranked and rolling aggregations with analytical SQL functions

2. Dataset & Setup Instructions

This case study uses the Supply Chain dataset from Kaggle. Follow the steps below to set up your environment before writing any queries.

2.1 Primary Dataset

Dataset Name: Supply Chain Analysis — DataCo Global

Source: Kaggle.com (Public Dataset)
Direct Link: <https://www.kaggle.com/datasets/shashwatwork/dataco-smart-supply-chain-for-big-data-analysis>
Format: CSV (import into MySQL, PostgreSQL, or SQLite)
Records: ~180,519 rows across 53 fields

2.2 Alternative / Supplementary Datasets

Dataset	Link	Use Case
Supply Chain Shipment Pricing	https://data.humdata.org/dataset/supply-chain-shipment-pricing-data	Freight & pricing analysis
Global Superstore Orders	https://www.kaggle.com/datasets/rohitsahoo/sales-forecasting	Order & region analysis
Inventory Management	https://www.kaggle.com/datasets/blastchar/telco-customer-churn	Stock-level exercises

2.3 Environment Setup Steps

- **Step 1:** Create a free Kaggle account at <https://www.kaggle.com>
- **Step 2:** Download the dataset ZIP and extract the CSV file
- **Step 3:** Open your SQL client (MySQL Workbench / pgAdmin / DB Browser for SQLite)
- **Step 4:** Create a new database named supply_chain_db
- **Step 5:** Import the CSV into a table named supply_chain using the import wizard
- **Step 6:** Verify by running: `SELECT COUNT(*) FROM supply_chain;`

SQL

```
-- Verify your import was successful
SELECT COUNT(*) AS total_rows FROM supply_chain;

-- Preview the first 10 rows
SELECT * FROM supply_chain LIMIT 10;

-- Check column names
DESCRIBE supply_chain;    -- MySQL
-- OR
SELECT column_name FROM information_schema.columns
WHERE table_name = 'supply_chain';    -- PostgreSQL / SQLite
```

3. Key Columns Reference

The main supply chain table contains columns across several business domains. The most important ones for your queries are listed below.

Column Name	Data Type	Description
Order Id	INT	Unique order identifier
Order Date	DATE	Date the order was placed
Ship Date	DATE	Date the order was shipped
Order Status	VARCHAR	COMPLETE, PENDING, CLOSED, CANCELED
Product Name	VARCHAR	Name of the product ordered
Category Name	VARCHAR	Product category
Department Name	VARCHAR	Business department
Sales	DECIMAL	Total sales amount (USD)
Order Item Quantity	INT	Units ordered
Order Item Discount	DECIMAL	Discount applied (0.0 – 1.0)
Order Item Profit Ratio	DECIMAL	Profit margin ratio
Shipping Mode	VARCHAR	Standard, First Class, Second Class, Same Day
Customer Id	INT	Unique customer identifier
Customer Segment	VARCHAR	Consumer, Corporate, Home Office
Order Region	VARCHAR	Geographic region of the order
Order Country	VARCHAR	Country of the delivery
Latitude / Longitude	DECIMAL	Delivery coordinates
Days for shipping (real)	INT	Actual days taken to ship
Days for shipment (scheduled)	INT	Planned days for shipping
Late_delivery_risk	INT	1 = at-risk delivery, 0 = on time

4. SQL Questions & Tasks

Answer each question with a complete, working SQL query. Include comments in your code to explain your logic. Difficulty levels are marked: Easy (Green), Medium (Orange), Hard (Red).

Part A: Exploratory Data Analysis (EDA)

Q1. Total Orders by Status [Easy]

Write a query to count the number of orders for each Order Status. Order the results from highest to lowest count.

SQL

```
-- Q1: Count orders per status
SELECT `Order Status`,
       COUNT(*) AS total_orders
FROM supply_chain
GROUP BY `Order Status`
ORDER BY total_orders DESC;
```

Q2. Monthly Sales Trend [Easy]

Calculate the total sales and total orders for each month-year. Show months with the highest revenue at the top.

SQL

```
-- Q2: Monthly sales trend
SELECT DATE_FORMAT(`Order Date`, '%Y-%m') AS order_month,
       COUNT(DISTINCT `Order Id`) AS total_orders,
       ROUND(SUM(Sales), 2) AS total_sales
FROM supply_chain
GROUP BY order_month
ORDER BY total_sales DESC;
```

Q3. Top 10 Products by Revenue [Easy]

Find the top 10 products by total sales revenue. Include the product category and total units sold.

SQL

```
-- Q3: Top 10 products by revenue
SELECT `Product Name`,
       `Category Name`,
       SUM(`Order Item Quantity`) AS units_sold,
       ROUND(SUM(Sales), 2) AS total_revenue
FROM supply_chain
GROUP BY `Product Name`, `Category Name`
ORDER BY total_revenue DESC
LIMIT 10;
```

Q4. Customer Segment Analysis [Easy]

Calculate average order value, total sales, and order count for each customer segment.

SQL

```
-- Q4: Customer segment summary
SELECT `Customer Segment`,
       COUNT(DISTINCT `Order Id`) AS total_orders,
       ROUND(SUM(Sales), 2) AS total_sales,
       ROUND(AVG(Sales), 2) AS avg_order_value
FROM supply_chain
GROUP BY `Customer Segment`
ORDER BY total_sales DESC;
```

Part B: Supply Chain Performance Metrics

Q5. Late Delivery Rate by Shipping Mode [Medium]

Calculate the percentage of late deliveries for each shipping mode. Use the Late_delivery_risk flag. Which shipping mode has the worst performance?

SQL

```
-- Q5: Late delivery rate by shipping mode
SELECT `Shipping Mode`,
       COUNT(*) AS total_shipments,
       SUM(Late_delivery_risk) AS late_deliveries,
       ROUND(100.0 * SUM(Late_delivery_risk) / COUNT(*), 2) AS late_rate_pct
FROM supply_chain
GROUP BY `Shipping Mode`
ORDER BY late_rate_pct DESC;
```

Q6. Average Lead Time per Category [Medium]

Compute the average actual shipping days vs. scheduled shipping days for each product category. Identify categories where actual time exceeds scheduled time the most.

SQL

```
-- Q6: Lead time analysis by category
SELECT `Category Name`,
       ROUND(AVG(`Days for shipping (real)`), 2) AS avg_actual_days,
       ROUND(AVG(`Days for shipment (scheduled)`), 2) AS avg_scheduled_days,
       ROUND(AVG(`Days for shipping (real)`)) -
           AVG(`Days for shipment (scheduled)`), 2) AS avg_delay_days
FROM supply_chain
GROUP BY `Category Name`
ORDER BY avg_delay_days DESC;
```

Q7. Order Fill Rate by Region [Medium]

Find the fill rate (percentage of COMPLETE orders) for each Order Region. Rank the regions from best to worst performance.

SQL

```
-- Q7: Order fill rate by region
SELECT `Order Region`,
       COUNT(*) AS total_orders,
       SUM(CASE WHEN `Order Status` = 'COMPLETE' THEN 1 ELSE 0 END) AS
completed,
       ROUND(100.0 * SUM(CASE WHEN `Order Status` = 'COMPLETE' THEN 1 ELSE 0
END)
           / COUNT(*), 2) AS fill_rate_pct
FROM supply_chain
GROUP BY `Order Region`
ORDER BY fill_rate_pct DESC;
```

Q8. High-Discount Orders Impact [Medium]

Identify orders where the discount is greater than 20%. Calculate the average profit ratio for discounted vs. non-discounted orders. What pattern do you observe?

SQL

```
-- Q8: Discount impact on profit ratio
SELECT
  CASE WHEN `Order Item Discount` > 0.2 THEN 'High Discount (>20%)'
        ELSE 'Normal Discount (<=20%)' END AS discount_group,
  COUNT(*) AS total_orders,
  ROUND(AVG(`Order Item Profit Ratio`) * 100, 2) AS avg_profit_pct,
  ROUND(SUM(Sales), 2) AS total_sales
FROM supply_chain
GROUP BY discount_group;
```

Part C: Advanced SQL — CTEs, Subqueries & Window Functions

Q9. Rolling 3-Month Revenue [Hard]

Using a window function, calculate the 3-month rolling average of total revenue. This helps identify seasonal trends.

SQL

```
-- Q9: 3-month rolling average revenue (window function)
WITH monthly_sales AS (
  SELECT DATE_FORMAT(`Order Date`, '%Y-%m') AS order_month,
```

```

        ROUND(SUM(Sales), 2) AS monthly_revenue
    FROM supply_chain
    GROUP BY order_month
)
SELECT order_month,
       monthly_revenue,
       ROUND(AVG(monthly_revenue) OVER (
           ORDER BY order_month
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ), 2) AS rolling_3mo_avg
FROM monthly_sales
ORDER BY order_month;

```

Q10. Rank Products Within Each Category [Hard]

Use RANK() or DENSE_RANK() to rank products by total sales within each category. Show only the top 3 products per category.

SQL

```

-- Q10: Top 3 products per category using window function
WITH ranked_products AS (
    SELECT `Category Name`,
           `Product Name`,
           ROUND(SUM(Sales), 2) AS total_sales,
           RANK() OVER (
               PARTITION BY `Category Name`
               ORDER BY SUM(Sales) DESC
           ) AS sales_rank
    FROM supply_chain
    GROUP BY `Category Name`, `Product Name`
)
SELECT `Category Name`, `Product Name`, total_sales, sales_rank
FROM ranked_products
WHERE sales_rank <= 3
ORDER BY `Category Name`, sales_rank;

```

Q11. Identify Repeat Late-Delivery Regions [Hard]

Using a subquery or CTE, find all regions where late delivery rate exceeds the global average late delivery rate. Label them as underperforming regions.

SQL

```

-- Q11: Regions above average late delivery rate (subquery)
WITH region_stats AS (
    SELECT `Order Region`,

```

```

        COUNT(*) AS total_orders,
        ROUND(100.0 * SUM(Late_delivery_risk) / COUNT(*), 2) AS late_rate
    FROM supply_chain
    GROUP BY `Order Region`
),
global_avg AS (
    SELECT ROUND(AVG(late_rate), 2) AS avg_late_rate FROM region_stats
)
SELECT r.`Order Region`, r.late_rate,
       g.avg_late_rate,
       'UNDERPERFORMING' AS status
FROM region_stats r
CROSS JOIN global_avg g
WHERE r.late_rate > g.avg_late_rate
ORDER BY r.late_rate DESC;

```

Q12. Customer Lifetime Value (CLV) [Hard]

Calculate the Customer Lifetime Value: total revenue, total orders, and average order value per customer. Rank customers by CLV and return the top 20.

SQL

```

-- Q12: Customer Lifetime Value ranking
WITH customer_value AS (
    SELECT `Customer Id`,
           `Customer Segment`,
           `Order Country`,
           COUNT(DISTINCT `Order Id`) AS total_orders,
           ROUND(SUM(Sales), 2) AS lifetime_revenue,
           ROUND(AVG(Sales), 2) AS avg_order_value
    FROM supply_chain
    GROUP BY `Customer Id`, `Customer Segment`, `Order Country`
)
SELECT `Customer Id`, `Customer Segment`, `Order Country`,
       total_orders, lifetime_revenue, avg_order_value,
       RANK() OVER (ORDER BY lifetime_revenue DESC) AS clv_rank
FROM customer_value
ORDER BY clv_rank
LIMIT 20;

```

Part D: Open-Ended Analysis Tasks

For the following tasks, you must write the SQL query yourself from scratch. No starter code is provided. Think through the business question, identify the relevant columns, and build your query step by step.

Q13. Supplier Performance Dashboard [Hard]

Design a query that produces a supplier/department-level performance dashboard including: total orders, total revenue, average late delivery rate, average profit ratio, and most popular shipping mode per department.

Q14. Year-over-Year Sales Growth [Hard]

Calculate the year-over-year percentage change in total sales for each product category. Identify which categories are growing and which are declining.

Q15. Order Anomaly Detection [Hard]

Find orders where the actual shipping days exceed the scheduled days by more than 5 days AND the order is marked COMPLETE. How many such anomalies exist, and which regions have the most?

Q16. Profitability by Geography [Medium]

Write a query that shows total revenue, total profit (Sales * Order Item Profit Ratio), and profit margin percentage by Order Country. Filter to countries with more than 500 orders.

5. Deliverables & Submission Guidelines

#	Deliverable	Details
1	SQL Script File	A single .sql file named StudentName_SupplyChain.sql with all 16 queries
2	Query Results	Screenshots or exported CSVs for each query result
3	Analysis Report	A 1–2 page summary answering: What insights did you find? What surprised you?
4	Comments in Code	Each query must have at least 2–3 comment lines explaining the logic

Grading Criteria

Correctness (40%) | Code Quality & Comments (20%) | Query Optimization (20%) | Analysis Report (20%)

Academic Integrity Notice

You may use reference materials and SQL documentation, but all queries must be independently written. Do not copy queries from online sources or share solutions with classmates.

6. Hints & SQL Reference

6.1 Useful SQL Functions

Function / Clause	Purpose	Example
COUNT(DISTINCT col)	Count unique values	COUNT(DISTINCT `Order Id`)
SUM() / AVG()	Aggregate totals/averages	SUM(Sales), AVG(`Days for shipping (real)`)
DATE_FORMAT(date, fmt)	Extract month/year	DATE_FORMAT(`Order Date`, '%Y-%m')
CASE WHEN ... END	Conditional logic	CASE WHEN Late_delivery_risk=1 THEN 'Late' END
WITH cte AS (...)	Common Table Expression	Breaks complex queries into readable steps
RANK() OVER (...)	Window ranking	RANK() OVER (PARTITION BY col ORDER BY val)
ROWS BETWEEN n PRECEDING	Rolling window	ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
CROSS JOIN	Join all rows of two tables	Used to compare each row to a global aggregate

6.2 Query-Building Strategy

1. Start simple: always write a basic SELECT first, then add complexity layer by layer.
2. Test intermediate steps: if using a CTE, run the CTE block alone first to check results.
3. Check NULLs: use IS NOT NULL in WHERE clauses if aggregates seem off.
4. Use LIMIT 10 while testing to avoid scanning all 180K rows every time.
5. Read the column names carefully — some have spaces and need backticks (``) in MySQL.
6. For percentages: multiply by 100.0 (not 100) to avoid integer division truncation.

6.3 Additional Learning Resources

- **SQL Window Functions Guide:** <https://mode.com/sql-tutorial/sql-window-functions/>
- **MySQL DATE Functions:** <https://dev.mysql.com/doc/refman/8.0/en/date-and-time-functions.html>
- **W3Schools SQL Reference:** <https://www.w3schools.com/sql/>
- **SQLZoo Practice Problems:** <https://sqlzoo.net/>
- **Supply Chain KPI Glossary:** <https://www.supplychaindigital.com/top10/top-10-supply-chain-kpis>

